OFFICIAL TECHNICAL STATUS REPORT

Brief! - Social Media Blog API

Application 1.0.2 | Report v2 | First run on real infrastructure

Project	Brief! - a full-stack web application: a message board where every post fits in 255 characters
Author	Ambroise Lumanikio
Application version	1.0.2 (revision B)
Report	OTSR v2 | October 3, 2026 | supersedes OTSR v1 (same day)
Stack	Java 11+, Javalin 5, JDBC, H2 | React 18, TypeScript, Vite | Maven (wrapper), JUnit 4 | OpenAPI 3 + Swagger UI
Overall status	**Running on the real stack.** Manual acceptance testing passed on Windows. Automated suite result not yet recorded (Section 3).

#	Section
1	Executive Summary
2	Changes Since OTSR v1
3	Test Results
4	How Brief! Works - Technology Guide
5	API Documentation (Swagger UI)
6	Security Posture
7	Known Limitations and Open Items
8	Product Directions, Milestones and Scaling
9	Roadmap
10	How to Run

© 2026 Ambroise Lumanikio | OneTechly

1. Executive Summary

OTSR v1 ended with one outstanding action: run the application against the real web framework and database, which the build environment could not do. That has now happened. On October 3 the application was built and started on Windows, served at `http://localhost:8080`, and exercised by hand with two accounts. Registration, sign-in, posting, deleting, the ownership rules and every error message behaved as designed.

Area	State	Status
Backend on real Javalin + H2	Compiles with 0 errors in VS Code; starts; serves API and web app on port 8080	PASS
Manual acceptance tests	11 scenarios across 7 screenshots, all as designed (Section 3)	PASS
Automated suite (67 tests)	Result of `.\mvnw.cmd test` has not been captured in a report yet	NOT RECORDED
Frontend	TypeScript; type-check clean; dev server and production build both verified	PASS
API documentation	New: Swagger UI at /docs/index.html, 12 operations	NEW
Name	Now written Brief! in the wordmark, tab title and startup message	DONE

One action still open

Manual testing shows the application works for the paths that were clicked. The 67 automated tests cover more: edge cases such as exactly 255 characters, malformed JSON, expired tokens and editing another user's post through the API directly. Run `.\mvnw.cmd test` once and record the `Tests run:` line in OTSR v3. Until then, treat the automated result as unknown, not as passed.

2. Changes Since OTSR v1

Change	Detail	Why
Maven Wrapper added (1.0.1)	mvnw, mvnw.cmd, .mvn/ in backend/	The development machine had no Maven installed; the wrapper downloads it on first use
Frontend language: TypeScript, then JavaScript, then TypeScript (1.0.0 - 1.0.2)	1.0.1 converted to .js/.jsx; 1.0.2 restored .ts/.tsx	The editor errors that prompted the conversion came from a missing npm install, not from TypeScript. The target job lists TypeScript
Name: Brief to Brief! (1.0.2 rev. B)	Wordmark, tab title, startup message, documents	Author's decision
API documentation page (1.0.2 rev. B)	frontend/public/docs/: openapi.json, index.html, Swagger UI files	Same convenience as the PixelPerfect /docs page
"API docs" link in the app	Under the server status line	Discoverability
Java source	One text change in Main.java (startup message). No logic changes since 1.0.0	

Lesson recorded: two rounds of confusion (red lines in the editor, `mvn` not found) were setup problems, not code problems. A README that states prerequisites and first-run commands explicitly prevents both; the README now does.

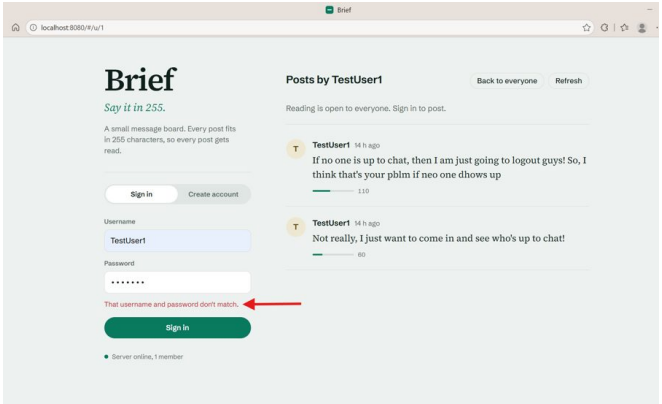
© 2026 Ambroise Lumanikio | OneTechly

3. Test Results

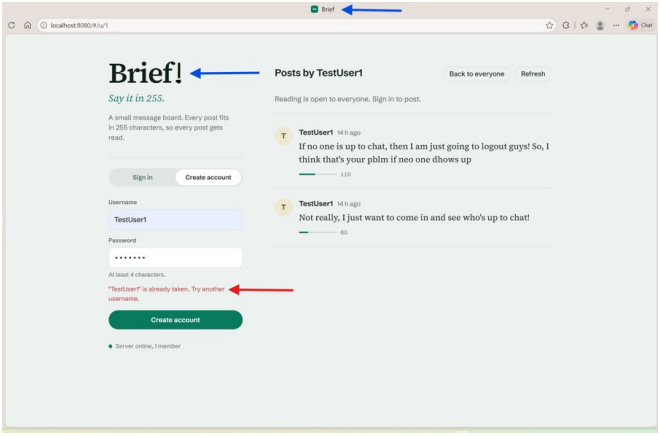
3.1 Manual acceptance testing (author, Windows, Microsoft Edge, `localhost:8080`, strict mode)

#	Scenario	Expected	Observed	Result
1	Create account TestUser1	Signed in; composer appears	"Signed in as TestUser1"; composer shown	**PASS**
2	Post two messages	Appear in feed, newest first, with length bar	Both shown with 110 and 60 characters	**PASS**
3	Sign in with a wrong password	Refused with a clear message (API 401)	"That username and password don't match."	**PASS**
4	Create account with a taken username	Refused (API 400)	"TestUser1 is already taken. Try another username."	**PASS**
5	Create account with a 3-character password	Refused before any request is sent	"Use a password of at least 4 characters."	**PASS**
6	Create account TestUser2	Member count becomes 2	"Server online, 2 members"	**PASS**
7	TestUser2 views TestUser1's posts	No Edit or Delete on someone else's posts	Only the length bar is shown on those posts	**PASS**
8	Delete own post, choose Keep	Post stays	Post stayed	**PASS**
9	Delete own post, choose Delete	Post removed	Post removed; "Your posts" shows the empty state	**PASS**
10	Multi-line post	Line breaks preserved	Shown on separate lines	**PASS**
11	Open a member's page (#/u/1) while signed out	Posts readable; sign-in required to post	"Reading is open to everyone. Sign in to post."	**PASS**

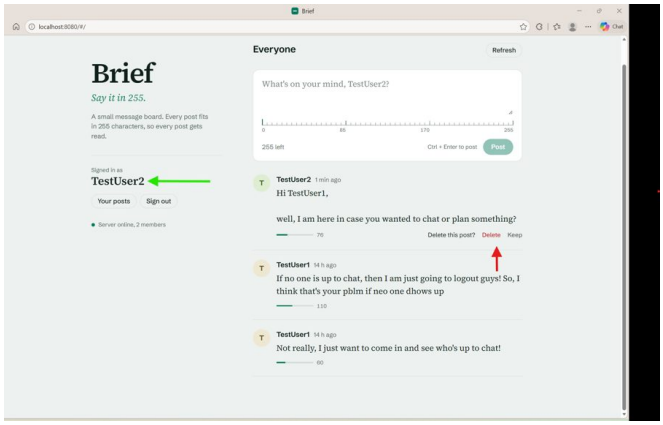
What these results prove beyond the screens themselves: the DAO SQL runs on a real database (accounts and messages are saved and read back), passwords verify through the hash, login tokens are issued and accepted (posting in strict mode needs one), and Javalin serves the web app and the API from one process. Those were the untested rows in OTSR v1.



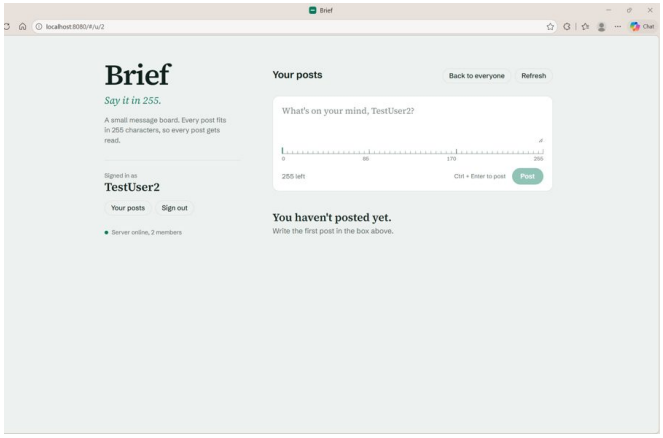
Scenario 3: wrong password.



Scenario 4: username already taken.



Scenarios 6-8: TestUser2 signed in; delete confirmation on own post; no actions on TestUser1's posts.



Scenario 9: after deleting, "Your posts" is empty.

3.2 Automated tests

Suite	Count	Where run	Result
unit.* (hashing, sessions, account and message rules)	29	Build environment, real classes	**PASS**
integration.ContractTest + StrictAuthTest	38	Build environment, against stand-ins for Javalin and the database	Pass, logic only
All 67 against real Javalin + H2	67	Author's machine: `.\mvnw.cmd test`	**NOT RECORDED**
Frontend type-check and build		Build environment and author's machine (dev server ran)	**PASS**
API docs page: loads, lists 12 operations, executes requests, shows the token header		Build environment, real browser, stand-in API	**PASS**

Note: if the jar was produced with `.\mvnw.cmd package`, Maven ran the tests first and would have stopped on any failure. If the app was started from VS Code's Run button instead, the tests did not run. Record which it was.

4. How Brief! Works - Technology Guide

This section explains the application for someone who built PixelPerfect and is now reading a Java + TypeScript codebase. The last column maps each piece to its PixelPerfect counterpart.

4.1 The pieces

Technology	What it is	What it does in Brief!	PixelPerfect equivalent
Java	Compiled, statically typed language	All backend code	Python
Javalin	Small web framework: maps URLs to Java methods	Routes in SocialMediaController	FastAPI
Jetty	The web server inside Javalin	Listens on port 8080	Uvicorn
Jackson	Converts between JSON and Java objects	Request bodies to Account / Message and back	Pydantic
JDBC	Java's standard way to talk to a SQL database	AccountDAO, MessageDAO	SQLAlchemy (lower level)
H2	A SQL database that runs inside the Java process	Stores accounts and messages in backend/h2/	PostgreSQL
Maven (+ wrapper)	Build tool: downloads libraries, compiles, tests, packages	pom.xml; mvnw.cmd	pip + requirements.txt
JUnit	Test framework	67 tests in src/test/java	pytest
React	Library for building the screen from components	App, Composer, MessageItem, AuthPanel, Ruler	React (same)
TypeScript	JavaScript plus declared types, checked before running	All frontend code (.ts, .tsx)	JavaScript (.js)
Vite	Dev server and bundler	npm run dev / npm run build	Create React App
OpenAPI + Swagger UI	A description of the API, and a page that renders it	/docs/index.html	FastAPI's automatic /docs

4.2 The three backend layers

Every request passes through three layers, each with one job. This separation is the main idea to be able to explain.

Layer	Question it answers	Example for "create a post"
Controller	What did the client ask for, are they allowed, and what status code goes back?	Read JSON, check the token, call the service, answer 200 / 400 / 401 / 403
Service	Is this allowed by the rules of the application?	Text not blank, at most 255 characters, author exists
DAO (Data Access Object)	How is it stored and fetched?	INSERT INTO message ... and return the row with its new id

4.3 What happens when you click Post

1. **Composer.tsx** checks the text is not blank and not over 255, then calls `onPost(text)`.
2. **App.tsx** `post()` calls `api.createMessage(session, text)`.
3. **api.ts** sends `POST /messages` with the JSON body and the header `Authorization: Bearer <token>`.

4. **SocialMediaController** `createMessageHandler` parses the JSON into a Message and asks `allowWriteAs`: is there a valid token, and is `posted_by` the caller?
5. **MessageService** `createMessage` applies the rules.
6. **MessageDAO** `saveMessage` runs the INSERT through a PreparedStatement and reads back the generated id.
7. The controller answers 200 with the saved message as JSON.
8. **App.tsx** puts the new message at the top of the list, React redraws the feed, and a "Posted" toast appears.

4.4 What happens when you sign in

1. AuthPanel.tsx sends `POST /login` with username and password.
2. AccountService looks up the account and compares the password with the stored **hash**. The database never holds the password itself, so the server re-computes the hash from what was typed and compares the two.
3. SessionService creates a random **token** and remembers which account it belongs to (for 12 hours).
4. The controller returns the account JSON, with the token in the `X-Auth-Token` response header.
5. api.ts reads that header; App.tsx keeps `{account_id, username, token}` in memory and in the browser's sessionStorage, so a page reload keeps you signed in until the tab closes. The password is not kept.

4.5 Where each rule is enforced

The browser checks rules to give fast feedback. The server checks them again because a browser can be bypassed - anyone can call the API directly. **The server check is the real one.**

Rule	Browser	Server
Post is 1-255 characters	Composer disables Post; ruler turns red	MessageService: 400
Password at least 4 characters	AuthPanel message	AccountService: 400
Username not already taken	(cannot know)	AccountService + unique column: 400
Must be signed in to write	Composer hidden when signed out	Controller allowWriteAs: 401
Only change your own posts	Edit / Delete hidden on others' posts	Controller allowWriteAs: 403

4.6 Glossary

Term	Meaning
DAO	Data Access Object. A class whose only job is SQL for one table.
DTO	Data Transfer Object. A class shaped for sending over the network. PublicAccount is one: an account without its password.
PreparedStatement	SQL with ? placeholders. User input is sent separately from the SQL text, which prevents SQL injection.
Connection pool	A set of open database connections that are reused instead of opened and closed for every query.
Hash and salt	A hash is a one-way scramble of the password. A salt is random data mixed in, so equal passwords get different hashes.
Bearer token	A random string that proves who you are. Whoever "bears" it is treated as that user, so it must stay secret.
Idempotent	Safe to repeat: doing it twice leaves the same result as doing it once. DELETE and logout behave this way.

© 2026 Ambroise Lumanikio | OneTechly

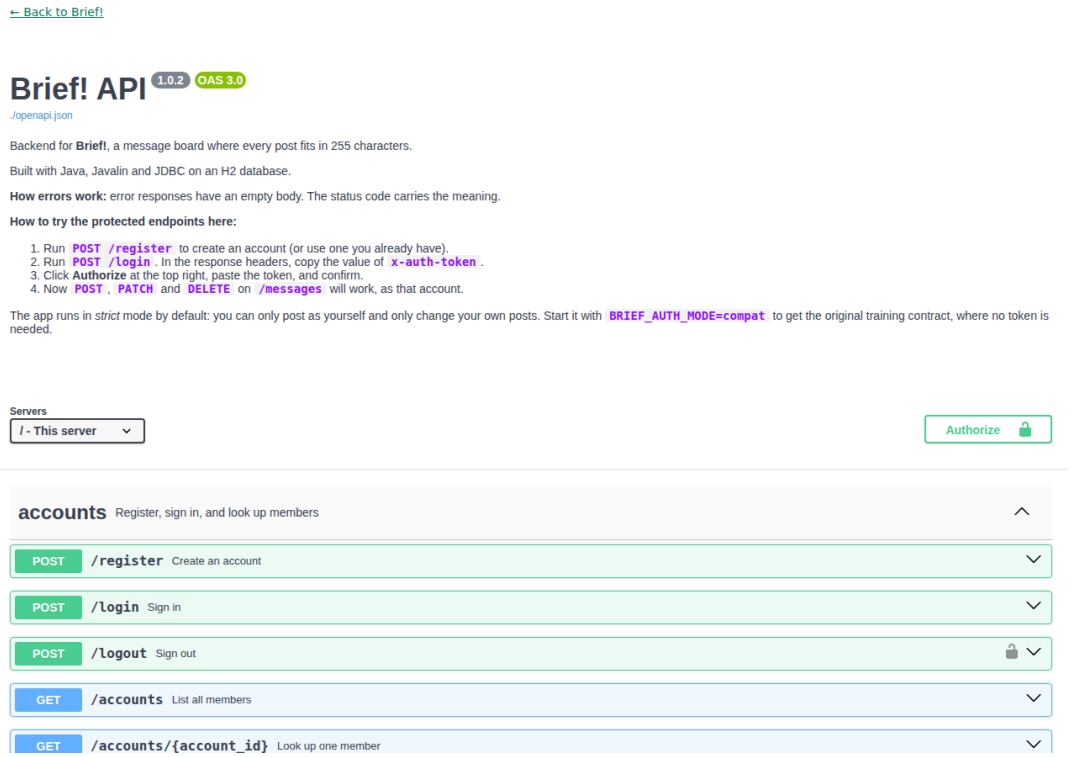
Term	Meaning
Strict / compat mode	Strict: tokens and ownership enforced (the running app). Compat: the original training contract with no tokens (used by the contract tests).
Proxy (Vite)	In development the browser talks to port 5173; Vite forwards API calls to port 8080.
SPA	Single-page application. One HTML page; JavaScript changes what is shown without reloading.
Interface / type (TypeScript)	A description of an object's shape, for example Message has message_id: number. Checked by the compiler, removed before the code runs.
Props and state (React)	Props are inputs a component receives from its parent. State is data a component remembers; changing it redraws the component.

© 2026 Ambroise Lumanikio | OneTechly

5. API Documentation (Swagger UI)

Swagger UI is not specific to Python. It displays any API described in the OpenAPI format. FastAPI writes that description automatically from the code; here it is a hand-written file, `frontend/public/docs/openapi.json`.

Item	Detail
Address	http://localhost:8080/docs/index.html (also linked as "API docs" in the app)
Content	12 operations in three groups: accounts (6), messages (5), operations (1), with request and response examples
Try it out	Sends real requests to the running server
Protected endpoints	Run POST /login, copy the x-auth-token response header, click Authorize, paste it
Hosting	Swagger UI's two files are served by the app itself - no internet connection needed
Maintenance cost	The file is not generated. When an endpoint changes, openapi.json must be edited to match, then npm run build



The API documentation page.

6. Security Posture

Topic	State	Status
Passwords at rest	PBKDF2-HMAC-SHA256, 120,000 iterations, random salt per password. Confirmed working on the real database (sign-in succeeds, wrong password fails)	DONE
SQL injection	PreparedStatement with bound parameters everywhere	DONE
Authorization	Server-side in strict mode. The UI behavior was confirmed by hand; the direct-API cases (403) are covered by StrictAuthTest	DONE
Session tokens	256-bit random, 12-hour lifetime, revoked on logout, held in memory	DONE
Password echoed in register / login responses	Required by the training contract; the web app discards it	ACCEPTED
HTTPS	None in the app. Required before any use outside localhost	OPEN
Login rate limiting	None	OPEN
Password rule	Minimum 4 characters (training contract). Too weak for real users	OPEN
Email verification, password reset	None; accounts have no email address	OPEN

© 2026 Ambroise Lumanikio | OneTechly

7. Known Limitations and Open Items

Priority	Item	Notes
CLOSED	Full stack not yet run on real Javalin and H2 (v1, HIGH)	Closed by Section 3.1
HIGH	Automated suite result not recorded	Run .\mvnw.cmd test; record the summary line in v3
MEDIUM	ConnectionUtil, pom.xml, SQL script and DAOs were written fresh, not taken from the training repository	They work (Section 3). Compare with the originals only if the original Revature tests must also pass
MEDIUM	Usernames are case-sensitive	"TestUser1" and "testuser1" would be two accounts. Decide and enforce one rule
MEDIUM	Sessions are lost when the server restarts	In-memory token store
MEDIUM	GET /messages returns every message	No pagination
MEDIUM	openapi.json is maintained by hand	Can drift from the code. A test that compares it with the registered routes would catch that
LOW	The version number 1.0.2 now covers two different builds	Revision B added the rename and the docs page. Use a new number for the next change
LOW	Feed does not update live	Refreshes on load, on window focus, and on Refresh
LOW	Web app has no automated tests	
LOW	Capitalized Java package names	Kept from the training template

8. Product Directions, Milestones and Scaling

Two directions were proposed: commercialize Brief!, or grow it into a general social network.

	A. General social network	B. Commercial product for a specific group
Who uses it	Anyone	For example: teams posting one short status update a day
Why they come	Because other people are already there	Because it solves a named problem for their own group
Hardest part	The empty-room problem: a network with no users has no value, and established networks are free	Finding the group and proving they will pay
Extra obligations	Content moderation, abuse reports, spam, legal requests - at any size	Smaller: private groups moderate themselves
Revenue	Advertising, which needs very large audiences	Subscription per team, which works at small scale
Fit with existing skills	New territory	Reuses the PixelPerfect playbook: tiers, Stripe billing, dashboards

© 2026 Ambroise Lumanikio | OneTechly

Assessment

Direction B, in a narrow niche. A private group does not need millions of users to be useful - five colleagues are enough - so the empty-room problem does not apply. The 255-character limit is a real feature for status updates: it forces a summary. A general social network is the harder path by a wide margin and is not recommended as a first goal.

Two cautions. First, PixelPerfect and ContentCalendar.ai already compete for the same hours; a third product needs a deliberate decision, not momentum. Second, "Brief" is a common word that other products already use. Check trademarks and domain availability before investing in the name.

Milestones (direction B)

Milestone	Goal	Contents
M0 Foundation	A link anyone can open	Record the test suite; Dockerfile; PostgreSQL; deploy behind HTTPS; stronger password rule; login rate limit
M1 Private groups	A team can use it alone	Workspaces, invitations by email, roles (owner, member); every post belongs to a workspace
M2 Daily habit	People come back	Replies or reactions, @mentions, a daily email digest, search, live updates
M3 Paid	First paying team	Stripe subscription per workspace, free tier limits, admin page, data export
M4 Trust	Ready for larger customers	Audit log, reporting and removal tools, single sign-on, backups and restore drills

Scaling: what breaks first, and the step that fixes it

Today	Breaks when	Step
H2 file database inside the app	More than one server, or real data worth protecting	PostgreSQL or MySQL; schema changes through Flyway migrations
GET /messages returns everything	A few thousand posts	Pagination by id ("give me 20 older than id X"); index on posted_by and time
Feed loads all members to show names	Many members	Return the author name with each message (SQL JOIN)
Tokens in memory	Second server instance, or any restart	Signed tokens (JWT) or a shared session store such as Redis
One Java process	Traffic or uptime needs	Stateless app in Docker, several instances behind a load balancer
Refresh to see new posts	Users expect live updates	Server-Sent Events or WebSocket
No limits	Abuse or scripts	Rate limits per account and per IP
Logs to the console	Problems you cannot see	Structured logs, request metrics, error alerts
Static files from the jar	Global users	CDN in front of the web app

© 2026 Ambroise Lumanikio | OneTechly

9. Roadmap

Version	Theme	Contents	Status
1.0.3	Close verification	Record the 67-test result; decide the username case rule; pagination on GET /messages	OPEN
1.1	Hardening	Login rate limit; stronger password rule in strict mode; CI running tests and the web build; a test that checks openapi.json against the routes	PLANNED
2.0	Spring Boot edition	Same contract on Spring Boot, Spring Data JPA (Hibernate) and MySQL; Spring Security with JWT; Bean Validation; springdoc generates the OpenAPI file automatically	PLANNED
2.1	Deployment (M0)	Dockerfile, docker-compose with the database, deployed behind HTTPS	PLANNED
3.x	Product (M1 - M3)	Only if direction B is chosen (Section 8)	PLANNED

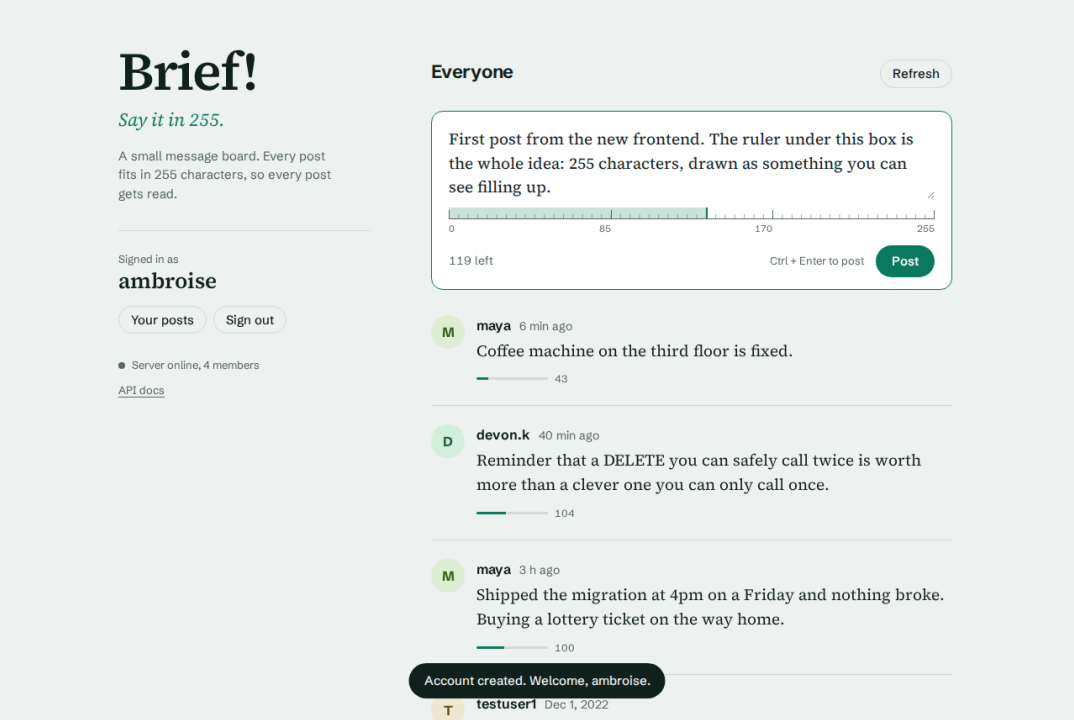
Version 2.0 remains the most valuable next step for the current goal: it is the same application on the stack the target role uses (Spring, Hibernate, MySQL), which makes a direct before-and-after comparison possible.

10. How to Run

Goal	Command (Windows PowerShell)
Run all tests	`cd backend; .\mvnw.cmd test`
Build the jar	`.\mvnw.cmd package`
Start the app	`java -jar target\brief-api.jar`
Open the app	http://localhost:8080
Open the API docs	http://localhost:8080/docs/index.html
Frontend in development	`cd frontend; npm install; npm run dev`
Rebuild the web app into the jar	`npm run build`
Start with an empty database	Stop the app, delete the backend\h2 folder

Two editions of 1.0.2 exist with identical behavior: the standard copy, and a study copy in which every file carries teaching comments. The compiled Java classes and the built web bundle of the two are byte-for-byte identical.

© 2026 Ambroise Lumanikio | OneTechly



Brief! 1.0.2 (revision B).